

Démarche, choix & axes d'amélioration

Une application de collecte quotidienne de l'état de forme de l'athlète d'endurance
— captant ce que seul l'humain sait, pour le confronter à la donnée objective.

Melvil Hamon • Application web (PWA) + back-end FastAPI • Standalone, branchable sur CoachAgent

1 Le problème

Un athlète ne vit jamais deux journées identiques. Un matin les sensations sont bonnes ; le lendemain, sans raison évidente, tout est plus lourd. Fatigue latente, accumulation invisible, impact des dernières séances, sommeil dégradé : ces signaux existent mais sont **diffus, difficiles à capter et à structurer**.

La montre mesure déjà très bien le *mesurable* (HRV, sommeil, charge). Ce qu'elle ignore, c'est ce que l'athlète **ressent** et **où ça tire**. J'ai donc conçu l'app autour d'un principe directeur :

« Ne demander à l'humain que ce que seul l'humain peut savoir. »

Le produit ne cherche pas à empiler une métrique de plus. Il capte le **subjectif** et le **localisé**, puis les confronte au moteur objectif. Le signal phare n'est pas une valeur isolée mais la **divergence subjectif ↔ objectif** — l'écart entre le ressenti et le mesuré, là où se cachent la fatigue masquée et le sur-risque de blessure.

Garde-fou assumé : toute sortie liée à une blessure est **une incitation à consulter un physio, jamais un diagnostic**.

2 La démarche

La friction d'abord

Une donnée quotidienne n'a de valeur que si elle est saisie **tous les jours**. Ma contrainte de conception n°1 est donc le temps de saisie : viser **~30 s, en gestes, sans clavier**. Le check-in repose sur des **swipe-cards** (une question, un geste) : ↑ bien plus · → un peu plus · double-tap « comme d'habitude » · ← un peu moins · ↓ bien moins. Le geste encode naturellement un *ressenti relatif* plutôt qu'une note absolue.

Trois sources, une lecture fusionnée

Subjectif

Check-in du jour, gênes localisées, mini-tests neuromusculaires.

Moteur

Modèle de Banister (fitness / fatigue / forme), ACWR, charge aiguë & chronique.

Objectif montre

HRV (RMSSD), sommeil, FC de repos, stress, body battery, VO₂max.

→ Readiness expliquée

Un score et les signaux qui l'ont déclenché — jamais un nombre opaque.

Une architecture en couches



Le moteur est abstrait derrière un `EnginePort` à deux implémentations : un **mode standalone** (snapshots locaux, zéro dépendance) et un **mode live** (REST vers CoachAgent). L'app reste démontrable seule tout en étant prête à se brancher sur l'écosystème Enduraw. Un générateur de **cohortes synthétiques déterministes** (personas + seed) rend les démos reproductibles et les règles testables.

Le mode *live* : l'intégration CoachAgent

À PROPOS · PROJET PERSONNEL

CoachAgent est mon projet personnel : un moteur qui **analyse les signaux de course à pied** — charge d'entraînement, fitness/fatigue façon Banister, ACWR, activités. Cette app Enduraw en est le **complément naturel** : elle capte ce que le moteur ne voit pas — le *subjectif et le localisé* (ressenti, gênes, mini-tests) — et le lui renvoie. Ensemble, ils ferment la boucle : **donnée objective + ressenti humain = une lecture de readiness plus juste**.

Concrètement, l'app dialogue avec CoachAgent via un client HTTP asynchrone (`httpx`) **caché derrière l' `EnginePort`** — le reste du code ignore quel moteur répond. Trois endpoints en lecture, deux en écriture (write-only) :

SENS	ENDPOINT	DONNÉE
Lecture	GET <code>/api/v1/engine/state</code>	forme / fitness / fatigue / ACWR du jour
Lecture	GET <code>/api/v1/engine/timeseries</code>	séries temporelles des métriques moteur
Lecture	GET <code>/api/v1/activities</code>	séances d'entraînement
Écriture	POST <code>/api/v1/wellness/daily</code>	check-in subjectif (+ compteur de gênes)
Écriture	POST <code>/api/v1/feedback/session</code>	RPE + affect post-séance

Robustesse. Authentification par *bearer token* ; timeout configurable (10 s par défaut) ; les erreurs transitoires (5xx, timeouts, erreurs de transport) sont **réessayées avec backoff exponentiel**, tandis que les 4xx échouent vite ; tout est mappé en **exceptions typées** (auth / introuvable / requête invalide / upstream) pour que l'app reste agnostique du backend. Les POST sont idempotents (*upsert*), donc sûrs à rejouer. En **mode standalone**, exactement la même interface est servie par un `MockEngine` (snapshots JSON) : l'app se démontre seule, sans CoachAgent.

3 Choix structurants (et pourquoi)

CHOIX	POURQUOI / ARBITRAGE
Échelles signées -2..+2 (forme, motivation, stress)	Mesurer un <i>écart à son propre normal</i> ancre la réponse sur l'athlète et neutralise le biais d'échelle — chacun n'a pas la même « note de 7/10 ».
Fatigue en 1..5 · stress optionnel	La fatigue se vit comme un niveau ; le stress ne doit pas alourdir la routine chaque jour. La friction prime sur l'exhaustivité.
Fusion à base de règles, pas de ML (MVP)	Explicabilité : on peut toujours dire <i>pourquoi</i> (les 2 signaux dominants). Pas de boîte noire, et démarrage sans gros historique.
Sync write-only (app → moteur)	L'app est la <i>source de vérité du subjectif</i> : elle pousse vers le moteur mais ne se laisse pas réécrire.
Readiness recalculée (stateless)	Pas de cache à invalider : la lecture du jour reflète toujours les dernières données.
Mini-tests dans l'app (saut CMJ, temps de réaction)	Des proxys <i>objectifs</i> captés par le téléphone, qui complètent la montre et révèlent une dégradation neuromusculaire avant le ressenti.
Design warm-black + accent flamme	Priorité lisibilité / contraste / mobile, avec états de chargement et états vides explicites.

Le moteur de readiness : 6 signaux, une cascade

Chaque règle produit un signal (déclenché ?, sévérité 0..1, évidence). Le score composite est leur moyenne pondérée ; la recommandation suit une cascade de priorité **consulter physio** → **repos** → **alléger** → **comme prévu**.

SIGNAL	POIDS	CE QU'IL DÉTECTE
divergence_subj_obj	0.30	Ressenti vs forme moteur ($\geq 1,5 \sigma$ d'écart) — l'ancre produit.
illness_hint	0.25	Motif maladie : HRV↓ + FC repos↑ + respiration↑.
niggle_escalation	0.20	Intensité d'une gêne en hausse (pente des reports).
niggle_load_correlation	0.15	Gêne ouverte après un pic d'ACWR ($> 1,3$).
wellness_divergence	0.15	HRV / sommeil / FC repos hors bande personnelle.
mini_test_trend	0.10	Saut ou réaction qui se dégrade vs baseline.

4 Les métriques présentées — et pourquoi celles-ci

L'app est un shell produit à **5 onglets**. Chaque écran répond à **une** question concrète de l'athlète. Le fil rouge : ne pas juger une performance, mais **détecter une dérive par rapport au normal** de l'athlète.

Accueil — « Est-ce que je dévie de mon normal ? »

Le check-in du jour comparé **aujourd'hui · hier · moyenne 7 j**, plus réaction et saut **exprimés en écart à la baseline**. *Pourquoi l'écart ?* « 290 ms » ne dit rien ; « +18 ms vs ta moyenne » dit immédiatement *tu es plus lent que d'habitude*. La dérive personnelle est le vrai signal.

MÉTRIQUE	ÉCHELLE	LECTURE
Forme ressentie	-2 . . +2	vs un jour normal
Fatigue	1 . . 5	1 = frais · 5 = épuisé
Motivation · Stress	-2 . . +2	vs un jour normal
Temps de réaction · Saut (CMJ)	ms · cm (écart)	dérive vs baseline perso

Charge — « Suis-je en train de surcharger ? »

VO₂max (échelle Garmin 5 bandes), ACWR avec **zone sûre 0,8–1,3** (référence du risque de surcharge), forme Banister (positif = frais).

Santé — « Comment je récupère ? »

Signaux objectifs de la montre : HRV/RMSSD (récup.), sommeil /100, body battery /100, stress /100 (plus bas = mieux).

Routine — « Mon check-in du jour »

Le cœur de la collecte : 4 swipe-cards puis 2 mini-jeux (réaction, saut). C'est ici que se joue la friction minimale.

Profil — « Globalement, où j'en suis ? »

La **synthèse en clair** (voir ci-dessous).

Profil : la synthèse en langage clair

Écran « Ton état de santé, en clair », volontairement **sans** reco entraîne/repos :

- **Indice du jour** : le score composite (0–100) traduit en mots — *Tout est au vert* (<34) · *Quelques signaux* (<67) · *Plusieurs signaux* (≥67). À 0, tout est dans la norme.
- **Tes ressentis vs tes données** : la divergence expliquée (« tu te sens mieux que tes données — attention à ne pas surcharger », ou « moins bien — sois patient, ça remonte »).
- **Signaux hors normes** : les signaux déclenchés du jour, avec leur évidence chiffrée.

Compte et réglages vivent derrière l'engrenage (écran dédié), pour garder le Profil centré sur l'**état de l'athlète**, pas sur l'admin.

5 État du dépôt (transparence)

Pour qu'aucune attente ne soit trompée, voici l'état réel — sans survente :

- ✓ **5 onglets en place** : Accueil, Charge, Routine, Santé, Profil.
- ✓ **Collecte subjective complète** : check-in signé, mini-tests, et tout le back-office gênes/blessures — modèle `Niggle / NiggleReport`, enum **BodyRegion (23 zones)** + côté, type de douleur, pattern mécanique, et API `/api/niggles`.
- ✓ **Fusion + sync** opérationnelles (`/api/insights/*`, `/api/sync/coachagent`), back-end testé.
- 🔴 **Saisie d'une gêne dans l'app** : l'écran existe mais reste accessible par URL et **n'est pas encore branché dans la navigation** — écran hérité en cours de portage dans le shell à 5 onglets. C'est exactement ce que l'axe ci-dessous vient renforcer.

6 Axes d'amélioration

★ PRIORITÉ PRODUIT

Maquette 3D du corps humain pour signaler une gêne

Le besoin. Le cœur de la thèse est « subjectif et localisé » : *où ça tire*. Aujourd'hui la région se choisit dans une liste de 23 zones — fonctionnel, mais lent et abstrait. Une **maquette 3D d'un corps humain** rendrait la saisie immédiate : l'athlète **tape directement la zone** qui le gêne, et **visualise** l'historique d'intensité en *heatmap* sur le corps (les zones qui « chauffent » = gênes récurrentes ou en escalade).

Approche technique. `react-three-fiber` + `dreii` ; un mesh de corps **segmenté par zones mappées 1-pour-1** sur l'enum `BodyRegion` **existante**, latéralité via l'enum `Side`. Point clé : **aucun changement du modèle de données** — on réutilise tel quel l'API `/api/niggles` et `NiggleReport` (intensité 0–10, type, pattern, timing). La 3D n'est qu'une **nouvelle vue** de saisie et de visualisation par-dessus un back-end déjà en place.

- 1 Rendre la carte du corps 2D pleinement interactive (zones cliquables → report) et la remettre dans la navigation.
- 2 Prototype 3D en lecture seule (rotation + zones colorées par intensité).
- 3 *Tap-to-report* sur le mesh 3D + heatmap historique.

Autres axes

- **ML sur la divergence** une fois l'historique suffisant : la fusion à base de règles reste le socle explicable, l'apprentissage l'affine.
- **Notifications de check-in** pour soutenir l'usage quotidien (le levier n°1 de la qualité des données).
- **Intégration Garmin réelle** en remplacement de l'ingestion simulée.
- **Baselines personnalisées** (fenêtres adaptées au profil de l'athlète).
- **Mode hors-ligne PWA complet** : saisie sans réseau, synchronisation différée.

Application web (PWA React + Tailwind) · back-end FastAPI / SQLAlchemy · fusion à base de règles · moteur abstrait (mock | CoachAgent). Détails techniques : [README.md](#) & [docs/](#) (architecture, modèle de données, contrat d'API, ADRs).